

TECHNICAL DOCUMENTATION

HeAlth

An AI health platform that turns a short questionnaire into a personalised weekly diet and exercise routine, grounded in Korea's 2024 national health statistics (국민건강통계).

Version 2.0 — single Next.js application with file-based storage
AI-Hackathon2026 · github.com/AI-Hackathon2026/HeAlth

Contents

- | | |
|----------------------------------|--|
| 1. Overview | 7. AI pipelines |
| 2. Architecture | 8. Authentication and security |
| 3. Project layout | 9. API reference |
| 4. Data storage | 10. Configuration |
| 5. Documents and statistics data | 11. Running and deploying |
| 6. Core logic | 12. Migration from v1 (Express + PostgreSQL) |

1. Overview

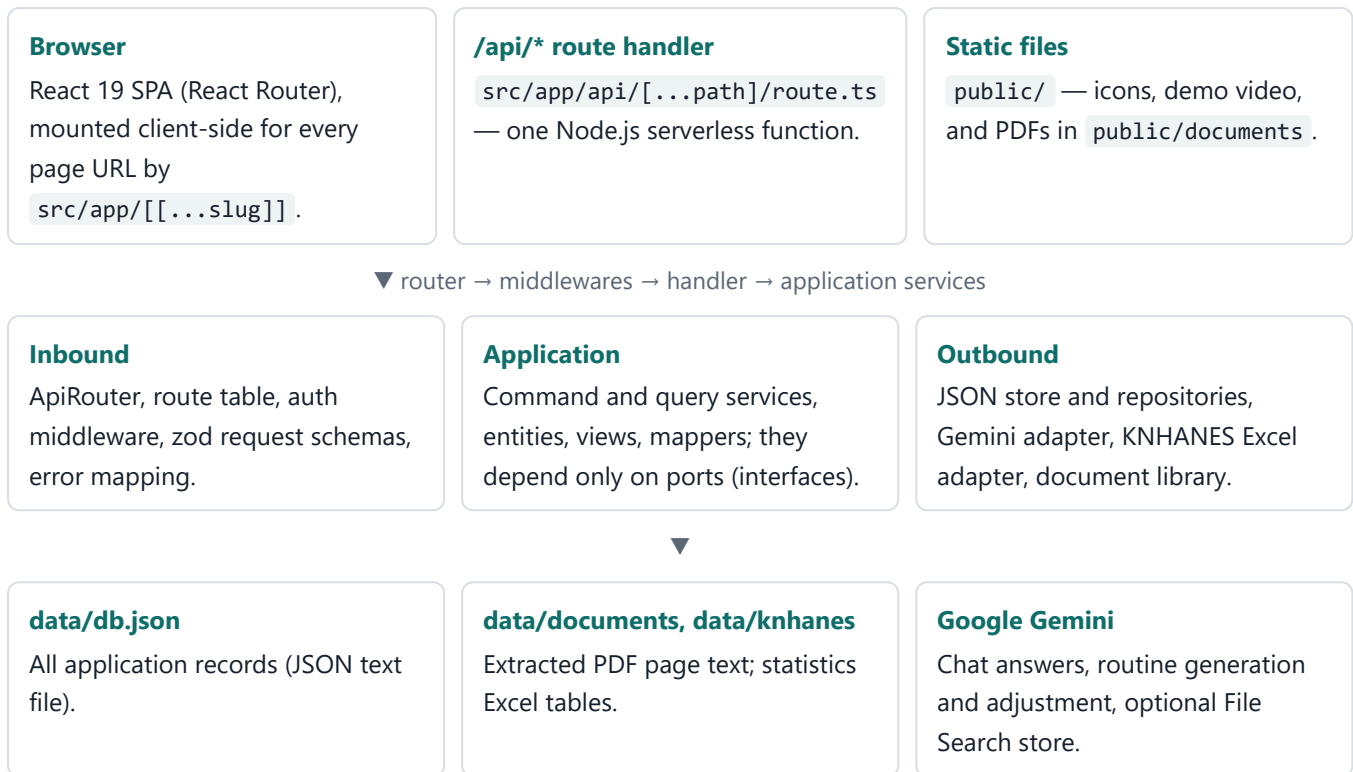
HeAlth guides a user through four steps:

1. **Check-in.** The user enters gender, age, height, weight and weekly drinking, smoking and exercise frequency.
2. **Assessment.** The server computes BMI, exposure rates for six risk areas, an overall health score (0–100), and a ranking against users of the same age group and gender.
3. **Routine.** The user picks a difficulty. Gemini generates a weekly plan with three meals per day (each food with calories) and exercises, plus a short summary and a Markdown report citing pages of the national statistics PDF.
4. **Follow-through.** The user ticks off meals and exercises, earns XP to level up a hero avatar, and chats with an AI coach that can rewrite parts of the routine.

Administrators manage the PDF library the AI reads from, browse documents in a page-flip viewer, and can query the KNHANES Excel tables directly.

2. Architecture

Everything runs in one Next.js 15 application. Browser pages and the JSON API share one origin, so authentication cookies are first-party and no CORS setup is needed.



The backend keeps the ports-and-adapters design of v1. Services receive repository interfaces through `src/server/container.ts`, so replacing PostgreSQL with a file only required new repository implementations. The business logic did not change.

3. Project layout

Path	Contents
<code>src/app/</code>	Next.js entry points: root layout, catch-all page, web manifest, API route.
<code>src/client/</code>	Former Vite frontend: <code>pages/</code> , <code>components/</code> (routine, avatar, chat, PDF viewer, admin), <code>api/client.ts</code> , hooks, utils, <code>styles/global.css</code> .
<code>src/server/Inbound/</code>	<code>http/router.ts</code> (Express-style path matching), <code>http/api.handler.ts</code> (request parsing, error mapping), <code>controller/routes.ts</code> (all endpoints), <code>controller/_communication/request</code> (zod DTOs).
<code>src/server/application/</code>	<code>command/services</code> , <code>query/services</code> , entities, views, mappers, <code>port/</code> interfaces.
<code>src/server/outbound/</code>	<code>store/</code> (JSON file database), <code>repo/</code> , <code>chatbot/gemini.ts</code> , <code>data/knhanes.adapter.ts</code> , <code>documents/document.library.ts</code> , <code>managers/scrypt-hash.manager.ts</code> .
<code>src/server/shared/</code>	Domain enums, exceptions, KNHANES rate tables, routine-generation and nutrition utilities, token/cookie/email/config utilities.
<code>scripts/</code>	<code>index-documents.ts</code> (PDF text extraction), <code>build-docs-pdf.mjs</code> (this document).

4. Data storage

`JsonStore` (`src/server/outbound/store/json.store.ts`) keeps the whole dataset in one pretty-printed JSON file at `DATA_DIR/db.json`.

- **Reads** return deep clones, so callers can never change stored data by accident.
- **Writes** run synchronously: mutate in memory, write a temp file, then rename it over `db.json`. A crash cannot leave a half-written file, and a write cannot interleave with another request in the same process.
- **Freshness:** the file's modification time is checked on every access, and the file is reloaded if another process changed it.
- **Relations:** the old foreign-key cascades are explicit helpers (`deleteUserCascade`, `deleteRoutineCascade`, `deleteChatCascade`). Unique constraints (email, one routine per user, one health profile per user) are checked in the repositories.
- **Transactions:** the `UnitOfWork` port is kept but runs the work directly. Each repository write is atomic on its own.

Collections

Collection	Key fields
users	id, email (unique), username, password (scrypt hash), role (ADMIN/USER), refreshToken (hash)
healthStatuses	userId (unique), gender, age, height, weight, alcoholFreq, smokeFreq, exerciseFreq
healthRecords	userId (unique), bmi, exposureRates {DISEASE: %}, overallScore
routines	userId (unique), difficulty, summary, report, reportReadme (Markdown), isActive
dailyRoutines	routineId, dayOfWeek (unique per routine), nutritionSummaryId
nutritionPlans / nutritionFoodItems	meal per day (mealType, calories, progress) / food per meal (name, calories, sortOrder, progress)
nutritionSummaries	vitamin A/C/D, calcium, protein, carbohydrates, fat, fiber, sugar, sodium, cholesterol (%)
exercisePlans	dailyRoutineId, exercises {task, frequency}, progressPercentage
chats / messages	userId, title, routineId (routine-coach chats) / chatId, role (USER/AI), text
characterProgress	userId (unique), level, xp, stage, heroStyle, totalCompletions
planProgressEvents	userId + planId + progressPercentage (unique): records XP already granted
uploadedFiles, hiddenDocumentIds	PDFs uploaded at runtime; bundled PDFs an admin removed from the library
ragStores, settings	Gemini File Search store name; selected Gemini model

Disease prevalence by exercise frequency (the v1 `DiseaseRateByExercise` table) was static seed data. It is now read directly from `src/server/shared/data/disease-rate.seed.ts`.

Vercel persistence. On Vercel the only writable location is `/tmp`, which belongs to a single function instance and is wiped when that instance is recycled. User data therefore lasts only for the life of an instance there. For persistent data, run `next start` on a host with a disk and set `DATA_DIR`.

5. Documents and statistics data

PDF library

- **Bundled PDFs** live in `public/documents/` (currently `2024 국민건강통계.pdf`, 343 pages) and are served as static files.
- `npm run index:documents` extracts each page's text with `unpdf` (`pdf.js`). It applies the same Korean text clean-up as `v1` and writes `data/documents/<name>.json`: `id` (derived from the filename), public URL, and page texts, where page 1 is index 0.
- **Uploaded PDFs** (admin, `POST /api/files/upload`) are stored as `DATA_DIR/uploads/<id>.pdf` with a `.pages.json` text file beside them.
- `DocumentLibrary` merges both sources. The admin PDF viewer asks `/api/files/download/:id` for a URL: the static file for bundled PDFs, or `/api/files/raw/:id` for uploads.

KNHANES tables

The 28 Excel workbooks of the 2024 국민건강통계 are in `data/knhanes/`. `KnhanesAdapter` finds the 2024 column and the row matching a metric and optional filters (sex, age, income). Only bare `.xlsx` filenames inside that folder are accepted.

6. Core logic

Health assessment

- $BMI = \text{weight(kg)} / \text{height(m)}^2$. Obesity status: ≥ 25 obese, < 18.5 underweight.
- Age groups: 19-29, 30-39, 40-49, 50-59, 60-69, 70+.
- Exposure rates (%) use 2024 KNHANES prevalence for the user's gender and age group. OBESITY is the full rate if $BMI \geq 25$, otherwise 30% of it. HYPERTENSION, DIABETES and STRESS use the group rate. SMOKING is 100 if the user smokes at all. ALCOHOL is 100 at 3+ times a week, 50 at 1-2, else 0.
- Overall score = $100 - \sum(\text{rate} \times \text{weight})$, with weights obesity 0.25, hypertension 0.25, diabetes 0.20, smoking 0.15, alcohol 0.10, stress 0.05.
- Ranking compares the score with every stored user of the same gender and age group. With no peers it is marked *estimated*.

Projections

For a disease, the projected rate at difficulty `d` is $\text{current} \times \text{rate}(\text{target days}) / \text{rate}(\text{current days})$. Target days are EASY 2, MODERATE 5 and HARD 7 days a week. Exercise frequency is snapped to the buckets 0, 1, 2, 3, 5 and 7.

Progress and gamification

- Food items, meals and exercises each have a progress value from 0 to 100; 100 means completed. A meal's progress is the aggregate of its food items.

- Each first completion of a plan item grants 25 XP. `planProgressEvents` prevents granting the same item twice.
- Level = floor(xp / 100) + 1, capped at 10. Stages: SPROUT (1–2), GROWTH (3–4), SKILLED (5–7), CHAMPION (8–9), MASTER (10). Level-ups are reported in the response so the UI can celebrate.
- Daily nutrition composition is computed on the server from the meal foods; Gemini is never asked for percentages.

7. AI pipelines

Health Q&A chat (`POST /api/chatbot`)

1. The question is reduced to a primary topic and expanded with Korean synonyms (e.g. 만성질병 ↔ 만성질환, 비만 ↔ 체중/BMI).
2. Every page of every library document that contains any term is scored. Term matches and statistics markers such as 유병률 or % raise the score; tables of contents and very short pages lower it. The top 6 pages are kept, 2,500 characters each.
3. With matches, Gemini answers in Korean (해요체) from that context and cites file and page. With none, it falls back to the Gemini File Search store, if one exists, or to a plain answer.
4. The question and answer are saved as messages of the chat.

Routine generation (`POST /api/routines/generate`)

1. Search terms are built from the user's highest exposure risks, and the top 10 matching PDF pages are retrieved.
2. A structured prompt with profile, risks and difficulty asks Gemini for JSON only: `report` , `reportReadme` and a `routine` entry per day with distinct menus and realistic per-food calories.
3. The JSON is validated and normalised (three meals per day, calories summed). If the README is missing, a fallback one is written. The result is stored as routine → daily routines → meal plans → food items, plus exercise plans and nutrition summaries.

Routine coach (`POST /api/routines/chat/:chatId/message`)

The current weekly routine is sent as JSON with the user's request. Gemini returns `userMessage` , an optional new `summary` , and `dayChanges` . Changed days have their meals merged (unchanged meals are kept) or their exercises replaced, and nutrition summaries are recomputed. Plain questions produce no changes.

Models

The default model is `gemini-2.5-flash` . `PATCH /api/chatbot/model` switches between the supported Gemini versions, and the choice is saved in `settings.geminiModel` . Without `GEMINI_API_KEY` , AI endpoints return 503 while the rest of the app keeps working.

8. Authentication and security

- Sign-in issues a JWT access token (15 min) and refresh token (7 days) in HTTP-only, SameSite=Lax cookies (Secure in production), plus a readable `csrfValue` cookie. The client retries once through `PATCH /api/auth/refresh` when a token expires.
- Refresh tokens are stored hashed and rotated on every refresh. Sign-out clears them.

- Passwords are hashed with scrypt (random 16-byte salt, 64-byte key, timing-safe comparison). v1 used reversible AES encryption.
- Roles: USER and ADMIN. File management, user listing and user deletion require ADMIN. Creating an admin needs an admin session or the `ADMIN_SIGNUP_KEY` header. The first admin comes from `ADMIN_EMAIL / ADMIN_PASSWORD`.
- Chats are owner-checked on read, update, delete and when sending chatbot messages.
- Error handling: business errors return their status code and message. Missing resources return 404 and unique conflicts 409. Unexpected errors return a generic 500, and are emailed to `EMAIL_USER` when SMTP is configured.

9. API reference

All paths are under `/api`. Bodies are JSON unless noted. "Auth" means a signed-in user; "Admin" means role ADMIN.

Auth and users

Method & path	Access	Request → response
POST /auth/signin	Public	{email, password} → {message, username, role}; sets cookies
POST /auth/signup	Public	{email, username, password} → {message, role}; sends notification emails if SMTP is set
PATCH /auth/refresh	Refresh cookie	→ {message}; rotates cookies
DELETE /auth/signout	Public	→ {message}; clears cookies
POST /auth/check-email	Public	{email} → 200 if an account exists, 404 otherwise
POST /users/signup/user	Public	{email, username, password} → 204
POST /users/signup/admin	Admin or key	{email, username, password} → 204
GET /users	Admin	→ [{id, email, username, role, createdAt, updatedAt}]
DELETE /users/:userId	Admin	→ 204 (cascades all of the user's data)
GET /users/me/character	Auth	→ {level, xp, xpInLevel, xpToNext, heroStyle, stage{key,name,emoji}, totalCompletions}
PATCH /users/me/character	Auth	{heroStyle: IRON DARK SPIDER} → character progress

Health

Method & path	Access	Request → response
POST /healthstatus	Auth	{gender, age 1–100, height, weight, alcoholFreq, smokeFreq, exerciseFreq (0–7)} → 201 assessment {bmi, obesity{status,label,guides}, lifestyle{guides}}
GET /healthstatus	Auth	→ stored health profile (404 if none)
PATCH /healthstatus/:id	Auth	same body → assessment; recomputes the health record
GET /health-records/me	Auth	→ {bmi, overallScore, exposureRates, healthRanking, formatted{overallScore, healthRanking, exposureRisks}}
GET /health-records/me/projection?disease=	Auth	→ {current, EASY, MODERATE, HARD, formatted{projections[]}}

Routines

Method & path	Access	Request → response
GET /routines/me	Auth	→ {summary, reportReadme, difficulty, days[{dayOfWeek, nutritionPlans[{foodItems[]}], exercisePlans[], nutritionSummary, averageCalories, isCompleted}], characterProgress}
POST /routines/generate	Auth	{difficulty: EASY MODERATE HARD} → 201 generated routine payload
PUT /routines/me	Auth	{difficulty} → replaces the routine with a new one
DELETE /routines/me	Auth	→ 204
PATCH /routines/exercise-plans/:id/progress	Auth	{isCompleted} or {progressionBar 0–100} → {id, isCompleted, progressionBar, leveledUp, previousLevel, characterProgress}
PATCH /routines/nutrition-plans/:id/progress	Auth	same as above, for a whole meal
PATCH /routines/nutrition-food-items/:id/progress	Auth	same as above, for one food; updates the meal's aggregate
POST /routines/me/chat	Auth	→ {chatId, routineId} (creates the coach chat if needed)
POST /routines/chat/:chatId/message	Auth	{text} → {aiResponse, routineUpdated, routine?}

Chat and chatbot

Method & path	Access	Request → response
GET / POST /chat	Auth	list chats / {title} → created chat (also available as /chats)
GET /chat/:chatId	Owner	→ {id, title, messages[{id, from, text, createdAt}]}
GET /chat/:chatId/messages	Owner	→ [{id, role, text, createdAt}]
PATCH / DELETE /chat/:chatId	Owner	{title} → updated / 204
PATCH / DELETE /chat/:chatId/messages/:messageId	Owner	{text} → updated / 204
POST /chatbot	Owner of chat	{chatId, text} → answer string
POST /chatbot/embedding	Auth	{text} → Gemini embedding vectors
GET /chatbot/available_models	Auth	→ string[]
GET / PATCH /chatbot/model	Auth	→ "You are using ..." / {model} → {message}

KNHANES and files

Method & path	Access	Request → response
---------------	--------	--------------------

GET /knhanes/files	Auth	→ {files: string[]}
POST /knhanes/query	Auth	{fileName, metric, filters?{sex,age,income}, userValue?} → {metricName, nationalAverage, source, userDeviation?}
POST /knhanes/ground	Auth	{sex?, age?, income?} → findings about meal skipping and nutrient intake
GET /files	Admin	→ [{id, filename, createdAt}]
POST /files/upload	Admin	multipart, field pdf (up to 4.5 MB on Vercel) → 204
GET /files/search?filename= · /files/find?keyword=	Admin	filename match / pages containing a keyword
GET /files/:id	Admin	→ {id, filename, createdAt, content: string[] (per page)}
GET /files/download/:id	Admin	→ {url, filename, expiresIn}
GET /files/raw/:id	Admin	→ application/pdf (uploads) or redirect (bundled)
DELETE /files/:id	Admin	→ 204 (bundled PDFs are hidden, uploads are deleted)
GET / DELETE /files/rag/store, /files/rag/documents	Admin	inspect or delete the Gemini File Search store and its documents

10. Configuration

Variable	Default	Purpose
TOKEN_SECRET	dev-only value	JWT signing secret; required in production
GEMINI_API_KEY	—	Enables AI features
ADMIN_EMAIL / ADMIN_PASSWORD / ADMIN_USERNAME	— / — / admin	First admin account, created on start-up
ADMIN_SIGNUP_KEY	—	Allows creating further admins through the API
ACCESS_TOKEN_EXPIRES_IN / REFRESH_TOKEN_EXPIRES_IN	15m / 7d	Token lifetimes
DATA_DIR	./data, or /tmp/health on Vercel	Location of db.json and uploads
EMAIL_USER / EMAIL_PASSWORD	—	Gmail SMTP for notifications

11. Running and deploying

```
npm install
cp .env.example .env.local      # fill in values
npm run dev                     # http://localhost:3000

npm run build && npm start       # production server
npm run index:documents         # after changing public/documents
npm run docs:pdf                # rebuild this document
```

Vercel: import the repository, set the environment variables, and deploy. No build settings need changing. The API route has a 60-second limit, which covers routine generation. The bundled PDF text and Excel tables are traced into the function through `outputFileTracingIncludes` in `next.config.ts`.

12. Migration from v1

v1 (two repositories)	v2 (this application)
Vite + React SPA on Vercel, proxying /api to an Express server on EC2	Next.js serves the same SPA and the API from one origin
Express controllers and routes	<code>ApiRouter</code> + <code>routes.ts</code> ; same paths, status codes and response shapes
PostgreSQL through Prisma (15 models)	JSON text file with the same collections
PDFs in S3, page text in the Content table	PDFs in <code>public/documents</code> , page text in <code>data/documents</code>
Redis, AWS SDK, multer, swagger, passport	Removed (unused or replaced)
AES-encrypted passwords	scrypt hashes
Open admin sign-up and user list	Admin-only, with an env-based bootstrap admin

The application layer (services, entities, routine generation, nutrition and scoring utilities) was carried over unchanged apart from import paths.